מבחן בקורס: עקרונות שפות תכנות, 202-1-2051
מועד: א
תאריך: 15/7/2025
שמות המרצים: מני אדלר, ירון גונן, יובל פינטר
מיועד לתלמידי: מדעי המחשב והנדסת תוכנה
משך המבחן: שלוש שעות
חומר עזר: אסור

הנחיות כלליות:

- יש לענות על כל השאלות בגיליון התשובות. מומלץ לא לחרוג מן המקום המוקצה.
- אם אינכם יודעים את התשובה, ניתן לכתוב 'לא יודע' על סעיף שלם ולקבל 20% מהניקוד.

שאלה 1:	תחביר וסמנטיקה _____	נק 30
שאלה 2:	מערכת טיפוסים _____	נק 20
שאלה 3:	תכנות פונקציונאלי, CPS, רשימות עצלות	נק 30
שאלה 4:	תכנות לוגי _____	נק 20
סה"כ	_____	נק 100

בהצלחה!

שאלה 1: תחביר וסמנטיקה [30 נקודות]

השפה L21 היא הרחבה של השפה L2, עם אפשרות להגדיר ערכי אתחול דיפולטיבים (ברירת-מחדל) לפרמטרים של פונקציה (למקרה שהם לא סופקו).
לדוגמא, התוכנית הבאה ב-L2:

```
(define f
  (lambda (x y)
     (/ 5 (+ x y))))
(f 9 1)
→ 0.5
```

ניתנת לכתיבה ב-L21 גם כ:

```
(define f
  (lambda (x (y 1))
     (/ 5 (+ x y))))
(f 9)
→ 0.5
```

או אף כ:

```
(define f
  (lambda ((x (+ 8 1)) (y (/ 2 2)))
     (/ 5 (+ x y))))
(f)
→ 0.5
```

אם נשלחים ארגומנטים עבור פרמטרים בעלי אתחול דיפולטי, הם כמובן יהיו הערכים של הפרמטרים בהפעלה:

```
(define f
  (lambda ((x 9) (y 1))
     (/ 5 (+ x y))))
(f 3 2)
→ 1
```

שימו לב, כי אם נשלחים חלק מהארגומנטים, הם נחשבים כמיועדים לפרמטרים הראשונים:

```
(define f
  (lambda ((x 9) (y 1))
     (/ 5 (+ x y))))
(f 4)
→ 1
```

א.

- עדכנו (בדף התשובות) את התחביר של L2 עבור L21. **שימו לב שהתחביר שהגדרתם תומך בקוד הנדרש בסעיף ב.**

```
<program> ::= (L21 <exp>+)              / Program(exps:List(exp))
<exp> ::= <define> | <cexp>             / DefExp | CExp
<define> ::= ( define <var-decl> <cexp> ) /
                                        DefExp(var:VarDecl, val:CExp)
<var-ref> ::= <identifier>              / VarRef(var:string)
<var-decl> ::= <identifier>   / VarDecl(var:string)
<cexp> ::= <number>                     / NumExp(val:number)
        |  <boolean>                    / BoolExp(val:boolean)
        |  ( lambda ( <var-decl>* ) <cexp>+ ) /
                                ProcExp(args:VarDecl[],body:CExp[]))
        |  ( if <cexp> <cexp> <cexp> ) / IfExp(test: CExp,
        |                                      then: CExp,
        |                                      alt: CExp)
        |  ( <cexp> <cexp>* )              / AppExp(operator:CExp,
        |                                          operands:CExp[]))
<prim-op>  ::= + | - | * | / | < | > | = | not |  eq?
<num-exp>  ::= a number token
<bool-exp> ::= #t | #f
```

- עדכנו את מימוש התחביר בקוד הפארסר (רק את מבנה הנתונים, לא את הפארסינג עצמו)

```
export type Exp = DefineExp | CExp;
export type AtomicExp = NumExp | BoolExp | StrExp | PrimOp | VarRef;
export type CompoundExp = AppExp | IfExp | ProcExp | LetExp | LitExp;
export type CExp =  AtomicExp | CompoundExp;

export type Program = {tag: "Program"; exps: Exp[]; }
export type DefineExp = {tag: "DefineExp"; var: VarDecl; val: CExp; }
export type NumExp = {tag: "NumExp"; val: number; }
export type BoolExp = {tag: "BoolExp"; val: boolean; }
export type PrimOp = {tag: "PrimOp"; op: string; }
export type VarRef = {tag: "VarRef"; var: string; }
export type VarDecl = {tag: "VarDecl"; var: string; }
export type AppExp = {tag: "AppExp"; rator: CExp; rands: CExp[]; }
export type IfExp = {tag: "IfExp"; test: CExp; then: CExp; alt:CExp;
}
export type ProcExp = {tag: "ProcExp"; args: VarDecl[], body:CExp[];
}
```

[4 נקודות]

ב. [13 נקודות]

ב.1 השלימו (בדף התשובות) את מימוש הפונקציה getAllRands1, המקבלת רשימת פרמטרים של פונקציה, רשימת ערכי אופרנדים עבור הפעלתה, וסביבה, ומחזירה את רשימת ערכי האופרנדים המלאה להפעלתה. כלומר, אם חסרים אופרנדים היא משלימה אותם על פי ערכי האתחול הדיפולטיביים של הפרמטרים, אם הוגדרו. במידה ויש בעיה כל שהיא היא מחזירה את רשימת האופרנדים המקורית.

```
const getAllRands1 = (params : VarDecl[], rands : Value[], env : Env)
: Value[] {
   if (rands.length >= params.length)
      return rands;
   else {
      const missingParams : VarDecl[] =
         params.slice(-(params.length - rands.length));
      //@TODO
   }
}
```

ב.2 השלימו (בדף התשובות) את מימוש הפונקציה getAllRands2, המקבלת רשימת פרמטרים של פונקציה ורשימת ביטויי אופרנדים עבור הפעלתה, ומחזירה את רשימת ביטויי האופרנדים המלאה להפעלתה. כלומר, אם חסרים אופרנדים היא משלימה אותם על פי ביטויי האתחול הדיפולטיביים של הפרמטרים, אם הוגדרו. במידה ויש בעיה כל שהיא היא מחזירה את רשימת האופרנדים המקורית.

```
const getAllRands2 = (params : VarDecl[], rands : CExp[]) : CExp[] {
   if (rands.length >= params.length)
      return rands;
   else {
     const missingParams : VarDecl[] =
        params.slice(-(params.length - rands.length));
        //@TODO
   }
}
```

ב.3 השלימו (בדף התשובות) את הפונקציה הבאה הממירה AppExp ב-L21 ל-AppExp שקול ב-L2. ניתן להניח שהאופרטור בהפעלה הוא ProcExp:

```
export const transApp  = (exp: AppExp) : AppExp
   //@TODO
}
```

ג. עבור כל אחד מארבעת (חלקי) המימושים הבאים של האינטרפרטר:

- ציינו האם מדובר במודל ההצבה או במודל הסביבות
- ציינו האם מדובר ב-applicative order או ב-normal order

- עדכנו את הקוד כך שייעשה שימוש בערכי האתחול הדיפולטיביים שהוגדרו לפרמטרים של הפונקציה, אם לא נשלחו עבורם אופרנדים.

[13 נקודות]

1.ג

```
const eval = (exp: CExp, env: Env): Result<Value> =>
    isNumExp(exp) ? makeOk(exp.val) :
    isBoolExp(exp) ? makeOk(exp.val) :
    isPrimOp(exp) ? makeOk(exp) :
    isVarRef(exp) ? applyEnv(env, exp.var) :
    isIfExp(exp) ? evalIf(exp, env) :
    isProcExp(exp) ? evalProc(exp, env) :
    isAppExp(exp) ? bind(eval(exp.rator, env), (rator: Value) =>
                        bind(mapResult(param =>
                              eval(param, env),exp.rands),
                               (rands: Value[]) =>
                                 applyProcedure(rator, rands, env)))
:
    makeFailure('Never');

const applyProcedure = (proc: Value, args: Value[], env: Env):
Result<Value> =>
    isPrimOp(proc) ? applyPrimitive(proc, args) :
    isClosure(proc) ? applyClosure(proc, args, env) :
    makeFailure(`Bad procedure ${format(proc)}`);

const applyClosure = (proc: Closure, args: Value[], env: Env):
Result<Value> => {
    const vars = map((v: VarDecl) => v.var, proc.params);
    const body = renameExps(proc.body);
    const litArgs : CExp[] = map(valueToLitExp, args);
    return evalSequence(substitute(body, vars, litArgs), env);
}
```

2.ג

```
const eval = (exp: CExp, env: Env): Result<Value> =>
    isNumExp(exp) ? makeOk(exp.val) :
    isBoolExp(exp) ? makeOk(exp.val) :
    isPrimOp(exp) ? makeOk(exp) :
    isVarRef(exp) ? applyEnv(env, exp.var) :
    isIfExp(exp) ? evalIf(exp, env) :
    isProcExp(exp) ? evalProc(exp, env) :
    isAppExp(exp) ? bind(eval(exp.rator, env),
```

```
        proc => applyProcedure(proc, exp.rands, env)) :

    makeFailure('Never');

const applyProcedure = (proc: Value, args: CExp[], env: Env):
Result<Value> => {
    if (isPrimOp(proc)) {
        const argVals: Result<Value[]> = mapResult((arg) =>
                                              eval(arg, env), args);
        return bind(argVals, (args: Value[]) =>
                                    applyPrimitive(proc, args));
    } else if (isClosure(proc)) {
        const vars = map((p) => p.var, proc.params);
        const body = renameExps(proc.body);
        return evalSequence(substitute(body, vars, args), env);
    } else {
        return makeFailure(`Bad proc applied ${format(proc)}`);
    }
};
```

3.λ

```
const eval = (exp: CExp, env: Env): Result<Value> =>
    isNumExp(exp) ? makeOk(exp.val) :
    isBoolExp(exp) ? makeOk(exp.val) :
    isPrimOp(exp) ? makeOk(exp) :
    isVarRef(exp) ? applyEnv(env, exp.var) :
    isIfExp(exp) ? evalIf(exp, env) :
    isProcExp(exp) ? evalProc(exp, env) :
    isAppExp(exp) ? bind(eval(exp.rator, env),
                      (proc: Value) =>
                        bind(mapResult((rand: CExp) =>
                           eval(rand, env), exp.rands),
                              (args: Value[]) =>
                                 applyProcedure(proc, args))) :
    makeFailure('Never');

const applyProcedure = (proc: Value, args: Value[]): Result<Value> =>
    isPrimOp(proc) ? applyPrimitive(proc, args) :
    isClosure(proc) ? applyClosure(proc, args) :
    makeFailure(`Bad procedure ${format(proc)}`);

const applyClosure = (proc: Closure, args: Value[]): Result<Value> =>
{
    const vars = map((v: VarDecl) => v.var, proc.params);
    return evalSequence(proc.body, makeExtEnv(vars, args, proc.env));
```

```
}
```

4.א

```
const eval = (exp: CExp, env: Env): Result<Value> =>
    isNumExp(exp) ? makeOk(exp.val) :
    isBoolExp(exp) ? makeOk(exp.val) :
    isPrimOp(exp) ? makeOk(exp) :
    isVarRef(exp) ? applyEnv(env, exp.var) :
    isIfExp(exp) ? evalIf(exp, env) :
    isProcExp(exp) ? evalProc(exp, env) :
    isAppExp(exp) ? bind(eval(exp.rator, env), (proc: Value) =>
                                applyProcedure(proc, exp.rands)) :
    makeFailure('Never');

const applyProcedure = (proc: Value, args: CExp[]): Result<Value> =>
    if (isPrimOp(proc)) {
        const argVals: Result<Value[]> = mapResult((arg) =>
                                                eval(arg, env), args);
        return bind(argVals, (args: Value[]) =>
                                  applyPrimitive(proc, args));
    } else if (isClosure(proc)) {
       const vars = map((v: VarDecl) => v.var, proc.params);
       return
          evalSequence(proc.body, makeExtEnv(vars, args, proc.env));
    } else {
        return makeFailure(`Bad proc applied ${format(proc)}`);
    }
}
```

שאלה 2: טיפוסים [20 נקודות]

נגדיר את השפה L51 כהרחבה של L5 לשפה שכוללת טיפוס חצי-פרימיטיבי חדש: **ביט** (`bit`), אשר לו שני ערכים `'none` ו-`'some` המכלילים את ההתנהגויות של הבוליאנים `#f` ו-`#t` עם אלה של המספרים `0` ו-`1`, כל אלה בהתאמה. בעת הפעלת פונקציות המצפות לבוליאן יתפרשו `'none` כ-`#f` ו-`'some` כ-`#t` (וכנ"ל בשאר סביבות בוליאן כמו מבחני `if`), ואילו בפונקציות המצפות למספר הם יתפרשו כ-`0` וכ-`1`, ללא החזרת שגיאה. בדומה, השפה תדע לפרש את הערכים `0`, `1`, `#t`, ו-`#f` כביטים במידת הצורך (casting).
בנוסף, השפה תאפשר להגדיר משתנים כביטים ולהגדיר פונקציות המצפות לביט או מחזירות ביט.

א. העניקו (בדף התשובות) טיפוסים במקומות הנדרשים לכך כך שהקוד הבא יעבור בדיקת טיפוסים ואינטרפרטציה:
[7 נקודות]

```
(define (abs : (number -> number))
    (lambda ((n : number)) : number
        (if (> n 0) n (- 0 n))))
(define (a : _____) 1)
(define (f : (_____ -> _____))
    (lambda ((x : _____)) : _____
        (abs (- x 1))))
(if (f a) 'success 'failure)
```

ב. עדכנו את הפונקציות הבאות בפארסר ובבודק הטיפוסים כדי לתמוך בצורה החדשה (אם נדרש), והוסיפו הגדרות בקוד לפי הדרוש (כלומר שהכל "יתקמפל" בסוף):
[13 נקודות]

ב.1 מתוך `TExp.ts`:

```
export type AtomicTExp = NumTExp | BoolTExp | StrTExp | VoidTExp;
export const isAtomicTExp = (x: any): x is AtomicTExp =>
    isNumTExp(x) || isBoolTExp(x) || isStrTExp(x) || isVoidTExp(x);
```

ב.2 מתוך `TExp.ts` (הפונקציה נקראת מתוך `checkEqualType` עבור המקרה `T.isAtomicTExp(te1) && T.isAtomicTExp(te2)`, ואם היא מחזירה `true`, כל הבדיקה תחזיר `makeOk(true)`):

```
export const eqAtomicTExp = (te1: AtomicTExp, te2: AtomicTExp):
boolean =>
   atomicTExpName(te1) === atomicTExpName(te2);
```

ב.3 מתוך L5-typecheck.ts:

```
export const typeofIf = (ifExp: IfExp, tenv: TEnv): Result<TExp> => {
    const testTE = typeofExp(ifExp.test, tenv);
    const thenTE = typeofExp(ifExp.then, tenv);
    const altTE = typeofExp(ifExp.alt, tenv);
    const constraint1 = bind(testTE, testTE =>
                        checkEqualType(testTE, makeBoolTExp(),
ifExp));
    const constraint2 = bind(thenTE, (thenTE: TExp) =>
                             bind(altTE, (altTE: TExp) =>
                                 checkEqualType(thenTE, altTE,
ifExp)));
    return bind(constraint1, (_c1: true) =>
                bind(constraint2, (_c2: true) =>
                    thenTE));
};
```

שאלה 3: תכנות פונקציונאלי, CPS, רשימות עצלות [30 נקודות]

א. כן / לא - צורת זנב [2 נקודות לכל ביטוי, סה״כ 10 נקודות]
עבור כל ביטוי, סמנו ״כן״ או ״לא״, האם הביטוי הוא בצורת זנב (tail form).
הניחו כי f, g, ו־h הן פונקציות משתמש.

ביטוי	צורת זנב?
`(f x)`	**כן / לא**
`(if (g x) (f y) (f z))`	**כן / לא**
`(cond (x (g x)) (else 0))`	**כן / לא**
`(+ 1 (f x))`	**כן / לא**
`(let ((y (f x))) y)`	**כן / לא**

ב. בהקשר של חישובים איטרטיביים ורקורסיביים, איזו מגבלה חלה על השפות L2 - L5 ללא תמיכה באופטימיזצית רקורסיית זנב? [5 נקודות]

_____

_____

_____

ג. רשימות עצלות: [7 נקודות]
השלימו את הפונקציה `make-range` אשר מקבלת שלושה פרמטרים: (1) ערך התחלתי, (2) ערך סופי ו-(3) גודל צעד, ומייצרת רשימה עצלה אשר האיברים בה הם מספרים מן הערך ההתחלתי עד הסופי, בקפיצות של גודל הצעד.
דוגמה:

```
(take (make-range 1 100 2) 3) ; => '(1 3 5)
```

בקוד שלכם, השתמשו ב-ATD עבור רשימות עצלות:

```
empty-lzl
empty-lzl?
cons-lzl
head
tail
```

```
;; [ Number * Number * Number -> Lzl[Number] ]
(define make-range
  (λ (start end step)
```

_____

_____

_____

_____

_____

_____

_____

ד. CPS

[7 נקודות]

השלימו את הקוד הבא עבור הפונקציה $sum-lzl-until אשר מקבלת שלושה פרמטרים: (1) רשימה עצלה של מספרים, (2) מספר מקסימלי, ו-(3) פונקציית המשך (continuation). הפונקציה מחשבת את סכום האיברים של הרשימה העצלה, עד שהיא מגיעה לאיבר גדול יותר מהמספר המקסימלי, ואז מעבירה את הסכום המצטבר לפונקציית ההמשך. דוגמה לחישוב יש בחתימת הפונקציה.

```
;; [Lzl[Number] * Number * [Number -> T1] -> T1 ]
;; Example: (sum-lzl-until$ ints 5 id) => 15
(define sum-lzl-until$
  (λ (lzl max cont)
     (if (empty? lzl)
          (cont 0)
```

_____

_____

_____

_____

_____

_____

_____

ה. מה הערך של הביטוי הבא? [1 נקודה]

```
(sum-lzl-until$ (make-range 1 100 2) 5 id)
```

_____

שאלה 4: תכנות לוגי [20 נקודות]

א. מצאו את ה-mgu של שני צמדי הביטויים הבאים. יש לפרט את צעדי החישוב.
[6 נקודות]

```
unify [ t(X,f(a),X),  t(g(U),U,W)  ]

unify[ t(X,f(X),X), t(g(U),U,W)  ]
```

ב. ממשו (בדף התשובות) את הפרוצדרות odd/1, even/1 עבור מספרי צ'רץ' זוגיים ואי-זוגיים בהתאמה:

```
% Signature: natural_number(X)/1
% Purpose: X is a natural number
natural_number(zero).
natural_number(s(X)) :- natural_number(X).

% Signature: even(X)/1
% Purpose: X is an even number.


% Signature: odd(X)/1
% Purpose: X is an odd number.
```

[6 נקודות]

ג. ממשו את הפרוצדורה split_odd_even(Nums, Odds, Evens), שהיא בעלת ערך אמת אם המספרים ברשימות Odds, Evens הם המספרים הזוגיים והאי-זוגיים בהתאמה ברשימה Nums

```
% Signature: split_odd_even(Nums,Odds,Evens)/3
% Purpose: Odds and Evens are a list of odd and even numbers
which compose Nums.
```

[6 נקודות]

ד. מהן התשובות לשאילתאות:

```
?-split_odd_even(Nums,[zero],[s(zero)])
?-split_odd_even(Nums,Odds,Evens)
```

[2 נקודות]